

Урок 3. Spring WebFlux Request Path поверх Reactor Netty

Как HTTP request после Reactor Netty попадает в Spring WebFlux и как результат controller-а превращается обратно в HTTP response?

0. Что слушатель поймет после урока

Конкретно, технически, после урока слушатель поймет:

- зачем нужен `ReactorHttpHandlerAdapter`;
- что такое `HttpHandler` и почему это нижний Spring HTTP-контракт;
- чем `HttpHandler` отличается от `WebHandler`;
- что такое `ServerWebExchange`;
- где в цепочке обработки запроса находятся `WebFilter`-ы;
- почему `DispatcherHandler` можно считать reactive-аналогом `DispatcherServlet`;
- как `HandlerMapping` находит нужный controller method;
- как `HandlerAdapter` физически вызывает controller method;
- что значит "controller возвращает `Mono/Flux`";
- что такое `HandlerResult`;
- зачем нужен `HandlerResultHandler`;
- где происходит подписка на `Publisher`;
- как результат controller-а превращается в HTTP status, headers и body;
- как response возвращается обратно в Reactor Netty и дальше в Netty `WRITE/FLUSH`.

Нас пока не интересует подробная механика:

```
flatMap
zip
retryWhen
publishOn
subscribeOn
...
```

Нас интересует именно путь HTTP-запроса внутри Spring WebFlux.

1. Где мы остановились на прошлом уроке

Концептуальная схема:

```
Client
↓
Netty server
↓
Channel
↓
ChannelPipeline
```

```

↓
EventLoopGroup / EventLoop
↓
reactiveBridge / Reactor Netty

```

Окей, Reactor Netty получил HTTP request.

Как этот request попадает в Spring WebFlux?

Кто вызывает слой Spring'a?

Кто создает Spring-абстракции request/response?

Где появляется `Handler`?

И как дальше дело доходит до `controller method`?

2. Концептуальная схема текущего урока

```

Reactor Netty
↓
ReactorHandlerAdapter
↓
Handler
↓
WebHandler / WebFilter chain
↓
DispatcherHandler
↓
HandlerMapping
↓
HandlerAdapter
↓
Controller returns Mono/Flux
↓
HandlerResult / HandlerResultHandler
↓
HTTP response write

```

Краткий итог вступления:

Прошлый урок: кто физически принял и прочитал байты запроса

Этот урок:

как Reactor Netty передает HTTP request в слой Spring WebFlux
и как Spring WebFlux доводит этот request до `controller method`.

Концептуальная схема перехода:

```

Netty world
  Channel
  ChannelPipeline
  ByteBuf
  read/write events
    ↓
Reactor Netty
    ↓
ReactorHandlerAdapter
    ↓

```

```

Spring world
  Handler
    ServerHttpRequest
    ServerHttpResponse
    ServerWebExchange
  WebHandler
    DispatcherHandler
    Controller

```

3. Reactor Netty: что это за слой

```

Netty
  работает с соединениями и байтами - дает низкоуровневую сетевую машину

Reactor Netty
  поднимает это до reactive HTTP request/response модели.

Spring WebFlux
  обрабатывает HTTP request как web-framework:
  фильтры, handler-ы, controller-ы и возвращает результат обработки.

```

4. ReactorHttpHandlerAdapter: мост из Reactor Netty в слой Spring'a

Теперь мы подошли к первой важной точке перехода

ReactorHttpHandlerAdapter — это адаптер, который позволяет Reactor Netty вызывать слой Spring WebFlux.

```

package org.springframework.http.server.reactive;

import reactor.netty.http.server.HttpServerRequest;
import reactor.netty.http.server.HttpServerResponse;

public class ReactorHttpHandlerAdapter implements BiFunction<HttpServerRequest, HttpServerResponse,
Mono<Void>> {

    ...

    private final org.springframework.http.server.reactive.Handler httpHandler;

    @Override
    public Mono<Void> apply(HttpServerRequest reactorRequest, HttpServerResponse reactorResponse) {
        ...
    }
}

```

На пальцах:

```

Reactor Netty говорит: "У меня есть HTTP request и HTTP response в моем формате".

Spring WebFlux говорит: "Я умею работать через свой Handler".

```

ReactorHandlerAdapter соединяет эти две стороны.

Концептуально:

```
Reactor Netty HttpServer
↓
Reactor Netty HttpServerRequest / HttpServerResponse
↓
ReactorHandlerAdapter
↓
Spring HttpHandler
```

Зачем нужен этот адаптер? ReactorHandlerAdapter

Spring WebFlux умеет работать не только поверх Netty. Он может быть адаптирован к разным server runtime. В Spring документации прямо сказано, что `HttpHandler` — это базовый контракт для обработки HTTP запросов, а рядом есть адаптеры: для Reactor Netty, Tomcat, Jetty и Servlet containers. То есть, Spring не хочет завязаться напрямую на Netty API.

5. HttpHandler: нижняя входная дверь Spring WebFlux

`HttpHandler` — это самый нижний HTTP-контракт Spring WebFlux.

На пальцах:

`HttpHandler` — это точка, куда Spring говорит:

"Дай мне HTTP request и HTTP response от слоя Netty, а я верну `Mono<Void>`, который завершится, когда обработка этого HTTP обмена будет закончена"

Сигнатура хэндлера:

```
package org.springframework.http.server.reactive;

public interface HttpHandler {

    Mono<Void> handle(
        org.springframework.http.server.reactive.ServerHttpRequest request,
        org.springframework.http.server.reactive.ServerHttpResponse response
    );
}
```

Как мы видим по сигнатуре, `HttpHandler` работает не с `Channel`, не с `ByteBuf`, и не с `Netty HttpServerRequest`. Он работает уже со Spring-абстракциями - `ServerHttpRequest` и `ServerHttpResponse`

Зачем нужен HttpHandler

После `ReactorHandlerAdapter` запрос попадает в Spring WebFlux через общий контракт

Мы можем писать приложения, используя

```

Reactor Netty
Tomcat
Jetty
Undertow
Servlet container

```

Но Spring хочет иметь общий вход:

```

Специфичный request/response
↓
adapter - ReactorHandlerAdapter
↓
Spring Handler

```

А значит, `Handler` - это граница, где Spring уже не думает категориями Netty. Spring начинает думать своими HTTP-абстракциями (`ServerHttpRequest`, `ServerHttpResponse`). А если точнее, это низкоуровневый Spring HTTP-контракт, говорящий: "Я умею выполнять HTTP request/response обмен", а значит, это вход в Spring reactive HTTP слой

Почему Mono

```

request пришел
↓
response будет записан в будущем
↓
когда запись response закончится, Mono<Void> завершится

```

Схема:

```

Handler.handle(ServerHttpRequest request, ServerHttpResponse response)
↓
Spring внутри запускает WebFlux chain
↓
результат controller-а будет записан в ServerHttpResponse
↓
когда запись завершится
↓
Mono<Void> completes - `onComplete`

```

То есть, `Handler` получает HTTP request(`ServerHttpRequest`) и HTTP response(`ServerHttpResponse`) в виде Spring-абстракций и возвращает `Mono`, который завершится тогда, когда обработка этого HTTP обмена будет закончена

И теперь, нам важно понять вот какую границу:

```

Reactor Netty
↓
ReactorHandlerAdapter
↓
Handler
↓
далее начинается Spring WebFlux processing chain

```

Еще раз, резюмируя про Mono

Почему `Handler` возвращает `Mono` ?

```
HTTP-обработка началась,
response будет записан через ServerHttpResponse,
а когда вся обработка и запись response завершится,
Mono<Void> завершится сигналом onComplete.
```

Если же произошла ошибка, цепочка завершится с `onError`.

То есть,

`Void` — потому что итоговое значение наружу не нужно.

`Mono` — потому что операция асинхронная.

6. WebHandler / WebFilter chain / ServerWebExchange слой

Мы дошли до границ

```
Reactor Netty
↓
ReactorHandlerAdapter
↓
Handler
↓
???
```

Следующий вопрос:

Окей, `Handler` получил спринговые `ServerHttpRequest` и `ServerHttpResponse`.

Но как из пары `http - request/response` получается полноценный `WebFlux`-контекст?

Где появляются `WebFilter`-ы?

Что такое `ServerWebExchange`?

И кто дальше передает запрос в `DispatcherHandler`?

То есть следующий микроблок строится на сущностях:

```
Handler
↓
ServerWebExchange
↓
WebFilter chain
↓
WebHandler
↓
DispatcherHandler
```

Важно: после `Handler` нам нужно смотреть не просто интерфейс, а реальную реализацию `Handler`, которая показывает переход выше к `ServerWebExchange`, и это класс

```
org.springframework.web.server.adapter.HttpWebHandlerAdapter
```

```

Reactor Netty
↓
ReactorHttpHandlerAdapter
↓
HttpHandler
↓
HttpWebHandlerAdapter
↓
DefaultServerWebExchange - что это?
↓
WebHandler / WebFilter chain - что это?

```

7. WebHandler / WebFilter chain / ServerWebExchange

До этого момента у нас было:

```

HttpHandler:

HttpHandler.handle(
    ServerHttpRequest request,
    ServerHttpResponse response
): Mono<Void>

```

Но для полноценного WebFlux-приложения пары request + response мало.

Spring нужно иметь не просто request и response, а единый контекст обработки запроса.

Этим контекстом является: `ServerWebExchange`

На пальцах:

```

ServerWebExchange — это контейнер одного HTTP-обмена внутри Spring WebFlux.

```

То есть это не только request.

Это целая связка:

```

ServerWebExchange
├── ServerHttpRequest
├── ServerHttpResponse
├── attributes
├── session
├── principal
├── form data
├── multipart data
└── другие данные текущего exchange

```

Концептуально:

```

request пришел
↓
создали ServerWebExchange — хранит в себе все о запросе
↓
дальше все WebFlux-компоненты работают с этим exchange

```

Что делает `HttpWebHandlerAdapter`

```
HttpWebHandlerAdapter.handle(request, response)
↓
createExchange(request, response)
↓
DefaultServerWebExchange
↓
delegate.handle(exchange)
↓
Mono<Void>
```

До него: `ServerHttpRequest` + `ServerHttpResponse`

После него: `ServerWebExchange`

8. Что такое `WebHandler`

`WebHandler` — следующий уровень после `HttpHandler`.

```
package org.springframework.web.server;

public interface WebHandler {

    Mono<Void> handle(org.springframework.web.server.ServerWebExchange exchange);
}
```

Разница:

```
HttpHandler: работает с request + response

WebHandler: работает с ServerWebExchange
```

То есть:

```
HttpHandler — нижний HTTP-вход.

WebHandler — веб-уровень Spring WebFlux, где запрос уже представлен как ServerWebExchange.
```

Где здесь `WebFilter chain`?

Правильная схема:

```
Reactor Netty
↓
ReactorHttpHandlerAdapter
↓
HttpHandler
↓
HttpWebHandlerAdapter
↓
WebHttpHandlerBuilder заранее собрал цепочку:
  WebFilter-1
  ↓
  WebFilter-2
```



```

↓
WebFilter-3
↓
DispatcherHandler
↓
DefaultServerWebExchange - что это?
↓
WebHandler / WebFilter chain - что это?

```

Важно уточнить:

```

WebHttpHandlerBuilder не выполняется на каждый request как бизнес-логика.

Он собирает инфраструктурную цепочку приложения при старте/инициализации.

А на каждый request HttpWebHandlerAdapter уже использует эту собранную цепочку.

```

Где смотреть в коде

```

1. WebHttpHandlerBuilder
2. HttpWebHandlerAdapter
3. FilteringWebHandler
4. DispatcherHandler

```

Главная ментальная модель:

```

DispatcherHandler — основной обработчик.

FilteringWebHandler — обертка над ним, которая прогоняет запрос через WebFilter-ы.

HttpWebHandlerAdapter — адаптирует это всё к HttpHandler и создает ServerWebExchange.

```

Краткий итог

WebFilter — это именно тот слой, где можно проверить Authorization header и не пустить запрос в controller, например.

HttpWebHandlerAdapter не создает фильтры. Он создает ServerWebExchange и вызывает уже подготовленный WebHandler - DispatcherHandler

9. WebHttpHandlerBuilder, HttpWebHandlerAdapter и WebFilter chain

```

1. WebHttpHandlerBuilder
  - собирает WebFlux processing chain из ApplicationContext;
  - находит главный WebHandler;
  - находит WebFilter beans;
  - находит WebExceptionHandler beans;
  - собирает итоговый HttpHandler.

```

2. `HttpWebHandlerAdapter`
 - является реализацией `HttpHandler`;
 - получает `ServerHttpRequest` + `ServerHttpResponse`;
 - создает `ServerWebExchange`;
 - запускает собранный `WebFlux chain`.
3. `ServerWebExchange`
 - общий контекст одного HTTP request-response обмена.
4. `FilteringWebHandler`
 - оборачивает целевой `WebHandler` цепочкой `WebFilter`-ов.
5. `WebFilter`
 - может пропустить request дальше через `chain.filter(exchange)`;
 - или завершить response и не пустить request в controller.
6. `DispatcherHandler`
 - целевой `WebHandler`, до которого запрос доходит после фильтров.

Схема текущего участка

```

ApplicationContext
├── DispatcherHandler bean - WebHandler
├── WebFilter beans
├── WebExceptionHandler beans
└── другие инфраструктурные beans
    ↓
WebHttpHandlerBuilder
    ↓
готовый HttpHandler
    ↓
HttpWebHandlerAdapter
    ↓
создает ServerWebExchange
    ↓
ExceptionHandlerWebHandler
    ↓
FilteringWebHandler
    ↓
WebFilter-1
    ↓
WebFilter-2
    ↓
DispatcherHandler
  
```

Блок далее

Мы уже разобрали:

```

Reactor Netty
↓
ReactorHttpHandlerAdapter
↓
HttpHandler
↓
HttpWebHandlerAdapter
↓
WebHttpHandlerBuilder заранее собрал цепочку:
WebFilter-1
  
```

```

↓
WebFilter-2
↓
WebFilter-3
↓
DispatcherHandler - главный requestHandler
↓
DefaultServerWebExchange - контекст запроса
↓
WebHandler / WebFilter chain - цепочка фильтров
↓
DispatcherHandler - главный requestHandler
↓
HandlerMapping
↓
HandlerAdapter
↓
Controller returns Mono/Flux
↓
HandlerResult / HandlerResultHandler
↓
HTTP response write

```

10. Как Spring собирает WebFlux processing chain

1. `WebHttpHandlerBuilder` собирает цепочку при старте приложения.
2. `DispatcherHandler` является главным `WebHandler`.
3. `FilteringWebHandler` оборачивает `DispatcherHandler` цепочкой `WebFilter`-ов.
4. `ExceptionHandlerWebHandler` оборачивает всё это цепочкой `WebExceptionHandler`-ов.
5. `HttpWebHandlerAdapter` адаптирует собранный `WebHandler` к `Handler` из предыдущего слоя и на каждый request создает `ServerWebExchange`.

Концептуально:

```

ApplicationContext
├── DispatcherHandler
├── WebFilter beans
├── WebExceptionHandler beans
└── other WebFlux infrastructure
    ↓
WebHttpHandlerBuilder
    ↓
HttpWebHandlerAdapter as Handler
    ↓
on each request:
    create ServerWebExchange
        ↓
        ExceptionHandlingWebHandler
            ↓
            FilteringWebHandler
                ↓
                WebFilter-1
                    ↓
                    WebFilter-2

```

↓
DispatcherHandler

11. DispatcherHandler — целевой WebHandler

В обычном annotation-based WebFlux-приложении главным обработчиком является:

```
org.springframework.web.reactive.DispatcherHandler
```

Его роль похожа на DispatcherServlet в Spring MVC.

DispatcherHandler — это главный WebHandler, который позже будет искать нужный controller method.

Он находится в Spring context как web handler, а WebHttpHandlerBuilder использует его как целевую точку обработки каждого входящего запроса

То есть: DispatcherHandler — основная цель цепочки, но перед ним могут стоять фильтры и обработчики ошибок.

DispatcherServlet и DispatcherHandler играют похожую архитектурную роль: они являются центральной точкой диспетчеризации запроса.

Но DispatcherServlet работает в Servlet/blocking stack, а DispatcherHandler работает в reactive WebFlux stack.

11.1 Что делает DispatcherHandler

Теперь нужно понять: Какой обработчик должен обработать этот запрос?

Например: `GET /test1`

Должен попасть в:

```
@GetMapping("/test1")
public Mono<ResponseEntity<Map<String, String>>> test1() {
    ...
}
```

Он работает через набор специальных компонентов:

```
HandlerMapping
HandlerAdapter
HandlerResultHandler
```

11.2 Общий алгоритм DispatcherHandler

```
DispatcherHandler получает ServerWebExchange
↓
спрашивает HandlerMapping: "Кто умеет обработать этот request?"
```

```

↓
HandlerMapping возвращает необходимый обработчик в виде Object - вернет Controller Bean вместе с необходимым
методом для вызова
↓
DispatcherHandler ищет подходящий HandlerAdapter: "Кто умеет вызвать этот handler?"
↓
HandlerAdapter вызывает handler/controller method
↓
controller возвращает Mono/Flux или другой результат
↓
результат упаковывается в HandlerResult
↓
DispatcherHandler передает HandlerResult в HandlerResultHandler
↓
HandlerResultHandler превращает результат в HTTP response

```

DispatcherHandler можно считать координатором в текущем контексте, который выполняет:

```

как искать @RestController methods;
как вызывать endpoints;
как отдавать resources;
как подготавливать аргументы controller method;
как обрабатывать все возможные return types;
как писать response body.

```

Для этого Spring WebFlux использует стратегию делегирования:

```

HandlerMapping:
    найти handler.

HandlerAdapter:
    вызвать handler.

HandlerResultHandler:
    обработать результат handler-а и записать response.

```

12 HandlerMapping

Мы только что уточнили важную мысль:

```

DispatcherHandler
↓
HandlerMapping находит handler
↓
HandlerAdapter вызывает handler

```

И поправим:

```

HandlerMapping обычно возвращает не просто объект controller-а, а handler.

Для @RestController это чаще всего HandlerMethod: controller bean + конкретный method.

```

8.1. HandlerMapping: кто находит нужный обработчик

HandlerMapping — это компонент Spring WebFlux, который отвечает за поиск обработчика для текущего HTTP-запроса. Это маршрутизатор внутри Spring WebFlux.

Он смотрит на текущий ServerWebExchange и пытается понять:

Какой handler должен обработать этот request?

HandlerMapping на основе URL запроса и множестве параметров типа:

- path;
- HTTP method;
- query params;
- headers;
- Content-Type;
- Accept;
- consumes;
- produces;
- другие request conditions.

HandlerMapping возвращает handler объект, являющийся сопоставлением Controller bean + method

Точнее, он вернет объект HandlerMethod

То есть структура вида:

```
HandlerMethod:
  bean = DemoController
  method = test1()
```

У DispatcherHandler есть список HandlerMapping. Он проходит по ним и спрашивает: Кто из вас может обработать этот ServerWebExchange?

Упрощенно:

```
DispatcherHandler
↓
HandlerMapping-1: "Я знаю handler для этого request?"
↓
HandlerMapping-2: "Я знаю handler для этого request?"
↓
HandlerMapping-3: "Я знаю handler для этого request?"
```

Первый подходящий HandlerMapping возвращает handler(HandlerMethod)

```
ServerWebExchange
↓
DispatcherHandler
↓
HandlerMapping list
↓
RequestMappingHandlerMapping
↓
HandlerMethod: DemoController#test1()
```

8.2. Главный HandlerMapping для @RestController

Для обычных annotation-based controllers главный интересующий нас класс:

```
org.springframework.web.reactive.result.method.annotation.RequestMappingHandlerMapping
```

Именно он работает с:

```
@RestController
@Controller
@RequestMapping
@GetMapping
@PostMapping
@PutMapping
@DeleteMapping
```

То есть он заранее, при старте приложения, сканирует controller beans и строит таблицу соответствий:

```
HTTP условия -> HandlerMethod
```

Например:

```
GET /test1
↓
DemoController#test1()

POST /users
↓
UserController#createUser(...)

GET /users/{id}
↓
UserController#getUser(...)
```

И при request-е он уже не “ищет все методы с нуля”. Он использует заранее подготовленную mapping-информацию.

Это важно:

Mapping-таблица строится при старте приложения.

На каждый request Spring только выбирает подходящий handler из уже известных mappings.

8.6. Что если handler не найден

Если ни один HandlerMapping не нашел обработчик, то Spring не может передать request в controller.

```
DispatcherHandler
↓
HandlerMapping-1: no handler
↓
HandlerMapping-2: no handler
↓
HandlerMapping-3: no handler
↓
```

```
handler не найден
↓
404 Not Found
```

То есть 404 появляется не потому, что controller “вернул 404”.

А потому что на этапе mapping-a Spring не нашел обработчик для этого request.

Главная мысль:

```
HandlerMapping отвечает только за поиск handler-a.

HandlerAdapter отвечает за вызов найденного handler-a.
```

Следующий вопрос:

```
Кто физически вызовет этот HandlerMethod,
подготовит аргументы метода
и получит результат controller-a?
```

9. HandlerAdapter

Мы уже дошли до места:

```
DispatcherHandler
↓
HandlerMapping
↓
HandlerMethod: DemoController#test1()
```

То есть Spring уже понял:

```
GET /test1 должен обработать DemoController#test1()
```

Но метод контроллера еще не вызван.

Теперь нужен компонент, который умеет физически вызвать найденный handler.

9.1. HandlerAdapter: кто вызывает найденный handler

HandlerAdapter отвечает за вызов handler-a, который был найден через HandlerMapping.

Для обычных @Controller / @RestController в WebFlux главный интересующий нас класс:

```
org.springframework.web.reactive.result.method.annotation.RequestMappingHandlerAdapter
```

Он умеет вызывать handler-ы типа HandlerMethod, то есть то, что нашел (HandlerMapping) RequestMappingHandlerMapping.


```

(handlerMapping) RequestMappingHandlerMapping
↓
нашел HandlerMethod: DemoController#test1()
↓
(handlerAdapter) RequestMappingHandlerAdapter
↓
вызвал DemoController#test1()

```

9.2. Что делает RequestMappingHandlerAdapter

Он не просто вызывает метод через reflection.

Перед вызовом controller method ему нужно подготовить аргументы.

Например, при:

```

@GetMapping("/users/{id}")
public Mono<UserResponse> getUser(
    @PathVariable String id,
    @RequestHeader("Authorization") String authorization,
    @RequestParam(required = false) String source
) {
    ...
}

```

Spring должен понять:

```

@PathVariable String id - взять из path /users/{id}

@RequestHeader("Authorization") - взять из headers

@RequestParam String source - взять из query string

```

То есть RequestMappingHandlerAdapter делает несколько важных вещей:

1. Берет найденный HandlerMethod.
2. Подготавливает аргументы controller method:
 - @PathVariable
 - @RequestParam
 - @RequestHeader
 - @RequestBody
 - ServerWebExchange
 - ServerHttpRequest
 - Principal
 - и другие поддерживаемые типы.
3. Вызывает controller method.
4. Получает return value.
5. Упаковывает результат в HandlerResult.

9.3. Что возвращает HandlerAdapter

После вызова controller method появляется результат. Например controller вернул:

```
Mono<ResponseEntity<Map<String, String>>>
```

RequestMappingHandlerAdapter не пишет response сам.

Он возвращает: HandlerResult

То есть:

```
HandlerAdapter вызвал controller
↓
controller вернул Mono<ResponseEntity<...>>
↓
HandlerAdapter упаковал это в HandlerResult
↓
DispatcherHandler передаст HandlerResult дальше в HandlerResultHandler
```

Главная мысль:

```
HandlerAdapter отвечает за вызов handler-a,
но не отвечает за финальную запись HTTP response.
```

9.4. Где мы сейчас в цепочке

```
DispatcherHandler
↓
HandlerMapping
↓
HandlerMethod: DemoController#test1()
↓
HandlerAdapter
↓
controller method physically invoked
↓
HandlerResult
```

То есть после HandlerAdapter controller уже вызван. Но результат еще не записан клиенту.

10. Controller returns Mono/Flux

Мы сейчас дошли до точки:

```
DispatcherHandler
↓
HandlerMapping
↓
HandlerMethod найден
↓
HandlerAdapter
↓
controller method физически вызван
↓
controller вернул результат
```

Теперь главный вопрос:

Что именно значит:
controller вернул Mono или Flux?

На этом этапе controller method уже действительно вызван.

Например:

```
@GetMapping("/test1")
public Mono<ResponseEntity<Map<String, String>>> test1() {
    return Mono.delay(Duration.ofSeconds(3))
        .map(ignored -> ResponseEntity.ok(Map.of("value", "Hello World")));
}
```

Важно: Вызов controller method ≠ готовый HTTP response.

Controller method в этом примере возвращает не готовый ResponseEntity, а Mono, то есть reactive Publisher.

На пальцах:

Controller возвращает не сам ответ,
а описание того, как этот ответ появится позже.

10.1. Assembly vs execution

Здесь надо ввести очень важное различие: assembly и execution

Assembly — это момент, когда мы собираем reactive chain.

Например:

```
Mono.delay(Duration.ofSeconds(3))
    .map(ignored -> ResponseEntity.ok(Map.of("value", "Hello World")));
```

В момент assembly мы не обязательно выполняем всю работу внутри цепочки.

Мы строим описание:

```
через 3 секунды испустить сигнал
↓
после этого выполнить map(...)
↓
создать ResponseEntity
```

То есть controller method быстро возвращает объект Mono.

```
controller method called
↓
Mono chain assembled
↓
Mono returned
↓
controller method finished
```

Execution — это момент, когда reactive chain реально начинает исполняться.

В Reactor цепочка обычно начинает исполняться после подписки:

```
subscribe
↓
signals start flowing
↓
onNext / onComplete / onError
```

То есть:

```
assembly — собрать pipeline
execution — запустить pipeline через subscription
```

10.2. Почему “вернул Mono” не значит “все уже выполнено”

Для такого кода:

```
@GetMapping("/test1")
public Mono<ResponseEntity<Map<String, String>>> test1() {
    log.info("controller method entered");

    return Mono.delay(Duration.ofSeconds(3))
        .map(ignored -> {
            log.info("creating response");
            return ResponseEntity.ok(Map.of("value", "Hello World"));
        });
}
```

Хронология такая:

```
t0:
HandlerAdapter вызывает controller method
controller method entered
controller возвращает Mono
controller method завершился

t0:
Spring позже подписывается на Mono
Mono.delay планирует timer

t0 + 3s:
delay испускает signal
map(...) выполняется
creating response
ResponseEntity создается
```

Главная мысль:

```
ResponseEntity создается не в момент входа в controller method.

ResponseEntity создается тогда,
когда reactive chain дойдет до map(...).
```

Для конкретно этой цепочки:

```
Mono.delay(Duration.ofSeconds(3))
    .map(ignored -> createResponse())
```

Ответ создается после delay.

10.3. Mono и Flux на уровне HTTP response

В WebFlux controller может вернуть Mono или Flux.

Mono — это publisher, который может дать: 0 или 1 значение

Смысл:

```
ответ появится позже, но максимум один объект
```

HTTP-сценарий:

```
Mono<UserResponse>
  ↓
UserResponse появляется
  ↓
Spring кодирует его в JSON
  ↓
пишет один HTTP response
```

Flux — это publisher, который может дать: 0..N значений

Например:

```
@GetMapping(value = "/events", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
public Flux<EventDto> events() {
    return eventService.events();
}
```

Смысл:

```
значения могут приходить много раз
```

HTTP-сценарий:

```
Flux<EventDto>
  ↓
event-1
  ↓
event-2
  ↓
event-3
  ↓
...
```

Для streaming response это может означать, что Spring будет писать данные в response постепенно, по мере прихода сигналов.

10.4. Controller не должен сам вызывать subscribe

В обычном WebFlux controller-е не нужно делать так:

```
@GetMapping("/bad")
public Mono<String> bad() {
    Mono<String> mono = service.call();

    mono.subscribe(value -> log.info("value={}", value));

    return Mono.just("ok");
}
```

Почему плохо:

```
ты вручную запускаешь отдельную цепочку;
Spring уже не контролирует ее lifecycle;
ошибка может потеряться;
response не связан с результатом этой цепочки;
backpressure/cancellation работают не так, как ожидается.
```

Правильно:

```
@GetMapping("/good")
public Mono<String> good() {
    return service.call()
        .doOnNext(value -> log.info("value={}", value));
}
```

То есть:

```
controller должен вернуть Publisher в Spring,
а Spring сам подпишется на него,
когда будет писать HTTP response.
```

Но подробный вопрос “где именно Spring подписывается” мы разберем позже, в блоке про `HandlerResultHandler`.

10.5. Что получает HandlerAdapter

Когда controller method возвращает `Mono<ResponseEntity<...>>`, `HandlerAdapter` получает именно return value.

То есть:

```
HandlerAdapter вызвал controller method
↓
получил Mono<ResponseEntity<Map<String, String>>>
```

Он делает такое:

```

берет return value
↓
создает HandlerResult
↓
отдает дальше

```

Схема:

```

controller returns Mono<ResponseEntity<...>>
↓
HandlerAdapter получает return value
↓
HandlerResult created
↓
HandlerResultHandler later processes it

```

Главная мысль:

```

Controller возвращает Publisher.
HandlerAdapter упаковывает этот результат в HandlerResult.
А подписка и запись response произойдут позже, на этапе HandlerResultHandler / при записи ответа(write).

```

11. HandlerResult

Мы уже прошли:

```

DispatcherHandler
↓
HandlerMapping
↓
HandlerMethod найден
↓
HandlerAdapter
↓
Controller method вызван
↓
Controller вернул Mono/Flux
↓
HandlerAdapter оборачивает результат контроллера в HandlerResult

```

Теперь вопрос:

```

Куда Spring кладет результат controller method
перед тем, как начать превращать его в HTTP response? - в HandlerResult

```

HandlerResult — это объект, который представляет результат вызова handler-a.

На пальцах:

```

HandlerResult — это упаковка вокруг того, что вернул controller method.

```

Например, controller вернул: `Mono<ResponseEntity<Map<String, String>>>`

Spring не сразу превращает это в HTTP response. Сначала HandlerAdapter упаковывает этот результат в HandlerResult.

По схеме:

```
RequestMappingHandlerAdapter
  ↓
вызывает DemoController#test1()
  ↓
controller returns Mono<ResponseEntity<Map<String, String>>>
  ↓
создается HandlerResult
```

11.1. HandlerResult еще не HTTP response

Например, если HandlerResult содержит: `return value = Mono<ResponseEntity<Map<String, String>>>`

Но это еще не значит, что:

```
HTTP status уже записан
headers уже отправлены
body уже закодирован в JSON
response уже ушел клиенту
```

На этом этапе Spring только говорит: контроллер вызван, результат получен, теперь нужно понять, кто умеет обработать этот результат.

Этим займется следующий участник: HandlerResultHandler

11.3. Что лежит внутри HandlerResult для нашего endpoint-a

```
@GetMapping("/test1")
public Mono<ResponseEntity<Map<String, String>>> test1() {
    return Mono.delay(Duration.ofSeconds(3))
        .map(ignored -> ResponseEntity.ok(Map.of("value", "Hello World")));
}
```

Когда RequestMappingHandlerAdapter вызовет этот метод, controller быстро вернет:

```
Mono<ResponseEntity<Map<String, String>>>
```

После этого будет создан HandlerResult.

Концептуально он содержит:

```
HandlerResult
├─ handler: DemoController#test1()
├─ return value: Mono<ResponseEntity<Map<String, String>>>
├─ return type: Mono<ResponseEntity<Map<String, String>>>
└─ context: данные, нужные для дальнейшей обработки результата
```

Но сам ResponseEntity еще не создан, потому что для этой цепочки:


```
Mono.delay(Duration.ofSeconds(3))
    .map(ignored -> ResponseEntity.ok(...))
```

Реальное значение появится только после подписки и после срабатывания delay.

То есть, HandlerResult уже существует, но значение внутри Mono еще не появилось.

11.4. Кто получает HandlerResult дальше

После создания HandlerResult управление возвращается в DispatcherHandler. Дальше DispatcherHandler должен найти подходящий HandlerResultHandler

Схема:

```
HandlerAdapter
  ↓
HandlerResult
  ↓
DispatcherHandler
  ↓
HandlerResultHandler
```

HandlerAdapter отвечает за вызов handler-a.

HandlerResultHandler отвечает за обработку результата handler-a.

12. HandlerResultHandler

Допустим, Controller уже вернул результат.

Кто понимает, что с этим результатом делать? Кто решает, как превратить Mono<ResponseEntity<...>> в HTTP response? - HandlerResultHandler

HandlerResultHandler — это компонент Spring WebFlux, который умеет обработать HandlerResult.

То есть:

```
HandlerAdapter:
  вызвал controller method
  получил return value
  создал HandlerResult

HandlerResultHandler:
  посмотрел на HandlerResult
  понял, что это за результат
  решил, как превратить его в HTTP response
```

12.1. Какие HandlerResultHandler бывают

Для обычных REST-контроллеров нам особенно важны result handlers, которые умеют работать с body и ResponseEntity.

Концептуально можно думать так:

```

ResponseEntityResultHandler:
  умеет обрабатывать ResponseEntity / HttpEntity / HttpHeaders.

ResponseBodyResultHandler:
  умеет обрабатывать @ResponseBody / @RestController return values.

ServerResponseResultHandler:
  умеет обрабатывать ServerResponse из functional endpoints.

ViewResolutionResultHandler:
  умеет обрабатывать view rendering.

```

12.2. MessageWriter: кто превращает body в JSON

Когда Spring понял, что body нужно записать, ему надо превратить Java-объект в байты HTTP response.

Для этого используются: `HttpMessageWriter`

`HttpMessageWriter` — это компонент, который умеет записать Java-объект в HTTP response body.

Для JSON обычно будет использоваться writer, который работает через Jackson.

Концептуально:

```

Map<String, String>
  ↓
HttpMessageWriter
  ↓
JSON
  ↓
DataBuffer
  ↓
ServerHttpResponse

```

То есть: `HandlerResultHandler` решает, как обработать return value. `HttpMessageWriter` отвечает за запись body в нужном формате.

12.3. Где здесь subscribe

Мы пока не уходим глубоко в этот блок, но важно подготовиться. Когда controller вернул `Mono<ResponseEntity<Map<String, String>>>`, кто-то должен подписаться на этот Mono, чтобы получить значение. Не controller. Не `HandlerAdapter`. А слой обработки результата и записи response.

Концептуально:

```

HandlerResultHandler
  ↓
начинает обработку return value
  ↓
обнаруживает Publisher
  ↓
организует подписку на Publisher
  ↓
получает сигналы
  ↓
по сигналам пишет HTTP response

```

Сейчас достаточно зафиксировать:

```
HandlerResultHandler — это место,
где результат controller-а начинает превращаться в HTTP response.
```

12.4. HandlerResultHandler еще не Netty

Важно не перескочить слишком низко. На этом этапе мы все еще в Spring WebFlux.

```
HandlerResultHandler
↓
HttpMessageWriter
↓
ServerHttpResponse
```

А уже потом:

```
ServerHttpResponse
↓
Reactor Netty response
↓
Netty ChannelPipeline
↓
WRITE / FLUSH
```

13. Где происходит subscribe

Это один из самых важных моментов всей лекции, потому что именно здесь соединяются две идеи: Controller вернул Publisher и HTTP response нужно реально записать клиенту

Подписка происходит внутри Spring WebFlux response handling слоя, когда Spring начинает записывать результат handler-а в HTTP response.

13.1. Почему вообще нужна подписка

Mono и Flux — это Publisher. Publisher сам по себе не обязан ничего делать, пока на него не подписались.

На пальцах:

```
Publisher — это источник будущих сигналов.

Subscriber — тот, кто говорит:
"Я готов получать сигналы".
```

Сигналы:

```
onSubscribe
onNext
onComplete
onError
```

Если controller вернул:

```
Mono<ResponseEntity<Map<String, String>>>
```

То Spring пока получил только объект Mono. Но чтобы получить из него реальный ResponseEntity, нужно подписаться.

```
Mono<ResponseEntity<...>>
  ↓
subscribe
  ↓
через 3 секунды onNext(ResponseEntity)
  ↓
onComplete
```

Controller возвращает Publisher в Spring.

Spring сам подписывается на него, когда приходит время получить значение и записать response.

Например:

```
@GetMapping("/test1")
public Mono<ResponseEntity<Map<String, String>>> test1() {
    return Mono.delay(Duration.ofSeconds(3))
        .map(ignored -> ResponseEntity.ok(Map.of("value", "Hello World")));
}
```

Хронология такая:

```
t0:
  controller вернул Mono<ResponseEntity<...>>

t0:
  Spring получил HandlerResult

t0:
  HandlerResultHandler начал response-handling

t0:
  Spring подписался на Mono

t0:
  только после подписки Mono.delay поставил timer на 3 секунды

t0 + 3s:
  Mono.delay испустил сигнал 0L

t0 + 3s:
  map(...) выполненся

t0 + 3s:
  ResponseEntity был создан

t0 + 3s:
  Spring получил onNext(ResponseEntity)

t0 + 3s:
  Spring разобрал ResponseEntity на status / headers / body
```

```

t0 + 3s:
  Mono испустил onComplete,
  то есть сообщил: "ResponseEntity больше не будет"

t0 + 3s:
  body пошел в HttpMessageWriter

t0 + 3s:
  JSON/DataBuffer записывается в ServerHttpResponse

t0 + 3s:
  ниже это уходит в Reactor Netty / Netty WRITE-FLUSH

```

13.2. Кто является Subscriber?

На уровне учебной модели можно сказать: Subscriber — это Spring WebFlux response-writing layer.

Subscriber находится внутри цепочки, которая пишет Publisher в ServerHttpResponse.

14. Response write path

Окей, Spring подписался на Publisher. Publisher испустил значение.

Как это значение превращается в HTTP response и возвращается обратно в Reactor Netty / Netty WRITE-FLUSH?

До этого момента цепочка выглядела так:

```

DispatcherHandler
  ↓
HandlerMapping
  ↓
HandlerAdapter
  ↓
Controller returns Mono/Flux
  ↓
HandlerResult
  ↓
HandlerResultHandler
  ↓
Spring подписывается на Publisher

```

```

@GetMapping("/test1")
public Mono<ResponseEntity<Map<String, String>>> test1() {
    return Mono.delay(Duration.ofSeconds(3)).map(ignored -> ResponseEntity.ok(Map.of("value", "Hello World")));
}

```

Допустим, теперь у Spring есть значение для ответа `ResponseEntity.ok(Map.of("value", "Hello World"))`

Но клиенту нельзя отправить Java-объект `ResponseEntity` напрямую.

Его нужно разобрать и превратить в обычный HTTP response.

Спринг достает

1. HTTP status
2. HTTP headers
3. HTTP body

А клиент ожидает не Java-объект, а байты HTTP-ответа.

Значит Body нужно закодировать. Чтобы отправить body клиенту, Spring должен превратить Java-объект в формат ответа.

Например:

```
Map.of("value", "Hello World")
```

Должен стать JSON:

```
{"value": "Hello World"}
```

Для этого в Spring WebFlux используется `HttpMessageWriter`

Концептуально:

```
Map<String, String>
  ↓
HttpMessageWriter
  ↓
  JSON
  ↓
DataBuffer
```

`DataBuffer` — это Spring-абстракция над буфером данных, которые можно записать в response.

То есть, спринг делает так:

```
Java object
  ↓
encoding
  ↓
DataBuffer
  ↓
ServerHttpResponse
```

14.1. `ServerHttpResponse.writeWith(...)`

Когда body превращен в поток `DataBuffer`, Spring должен записать его в HTTP response. Для этого используется `ServerHttpResponse`.

```
ServerHttpResponse.writeWith(Publisher<DataBuffer>) - Вот поток буферов с данными ответа. Запиши их в response.
```

`ServerHttpResponse` — это еще Spring-абстракция. На этом уровне мы все еще не вызываем напрямую `Netty Channel.write(...)`.

Spring пишет в свой `ServerHttpResponse`, а ниже эта абстракция связана с `Reactor Netty response`.

14.2. Возврат в Reactor Netty

После того как Spring начал запись в `ServerHttpResponse`, данные уходят ниже:

```
ServerHttpResponse
  ↓
Reactor Netty response
```

То есть мы возвращаемся из мира Spring WebFlux обратно в мир Reactor Netty.

Если в начале урока request шел вверх:

```
Reactor Netty
  ↓
ReactorHttpHandlerAdapter
  ↓
HttpHandler
  ↓
Spring WebFlux
```

То теперь response идет вниз:

```
Spring WebFlux
  ↓
ServerHttpResponse
  ↓
Reactor Netty response
  ↓
Netty ChannelPipeline
  ↓
Channel
```

14.3 Outbound ChannelPipeline

Когда Reactor Netty получает данные для записи, они уходят в Netty outbound path.

Вспоминаем прошлый урок:

ChannelPipeline работает в двух направлениях.

Inbound: bytes -> HTTP request -> Spring

Outbound: Spring response -> HTTP response -> bytes

Теперь мы как раз в outbound-направлении.

Концептуально:

```
DataBuffer / response data
  ↓
Reactor Netty
  ↓
Netty outbound ChannelPipeline
  ↓
HTTP encoder
  ↓
bytes
```

↓
Channel

На этом этапе HTTP response превращается в байты вида:

```
HTTP/1.1 200 OK Content-Type: application/json
...
{"value":"Hello World"}
```

14.4. WRITE / FLUSH

Финальный сетевой этап выполняет Netty/EventLoop.

Упрощенно:

```
WRITE - положить данные ответа в outbound-буфер Netty (Подготовить данные к отправке)
FLUSH - протолкнуть накопленные данные дальше в socket/OC (Реально отправить их дальше в сторону клиента.)
```

На уровне логов это может выглядеть так:

```
reactor-http-nio-3 WRITE: 94B
reactor-http-nio-3 FLUSH
```

Это значит:

EventLoop-поток Reactor Netty записал байты HTTP-ответа в Channel и протолкнул их дальше в socket.

16. Следующий урок — полная практика сегодняшнего урока с объяснениями. Reactive Streams Protocol

Дальнейшие темы следующего урока:

Темы:

- `Publisher`.
- `Subscriber`.
- `Subscription`.
- `request(n)`.
- `onSubscribe`, `onNext`, `onError`, `onComplete`.
- Backpressure.
- Cold vs hot.
- Lazy execution.
- Почему без subscription ничего не происходит.
- Почему `Mono` это 0..1, а `Flux` это 0..N.